プログラミングの基本

変数・代入・出力

- **変数宣言，代入**　`a = 10`, `a = b`
  - Cのような変数宣言は不要

```
int a = 10;
int b = 20;
a = b;
```

Cの記法

- **出力**　`print(a)`
  - 変数 a が記憶している**値**を画面上に表示
  - 「a」という**文字列**を表示したいときは，`print("a")`
  - 末尾に改行が入る
    - 改行を入れずに空白を入れる：`print(a, end=" ")`

```
printf("a¥n");
printf("%d¥n", a);
```

変数・代入・出力

- **サンプルコード**

```
a = 10
b = 20
a = b
print(a)
```

- **実行結果**
  - **20**

```
#include <stdio.h>
int main(void) {
    int a = 10;
    int b = 20;
    a = b;
    printf("%d¥n", a);
    return 0;
}
```

四則演算の書き方

- **足し算**：a + b
- **引き算**：a - b
- **掛け算**：a * b
- **割り算**：a / b
- **その他の計算**
  - a ÷ b の余り：　a % b
  - a ÷ b の整数部分：　a // b
  - a の b 乗：　a ** b

条件分岐 if

- 処理を**条件**により選択する.
  ⇒ 条件が**真(YES)**のときは処理群1，**偽(NO)** のときは処理群2を実行
- **インデント**(Tab)で処理群を区別

```
if 条件式:
Tab 処理群
elif 条件式:
Tab 処理群
else:
Tab 処理群
```

```
if (条件式) {
    処理群
} else if (条件式) {
    処理群
} else {
    処理群
}
```


条件式の書き方

- x と y の値が等しい　=　: x == y
- x と y の値が等しくない　≠　: x != y
- x が y より小さい　<　: x < y
- x が y 以下　≦　: x <= y
- x が y より大きい　>　: x > y
- x が y 以上　≧　: x >= y

条件分岐 if

- **サンプルコード**

```
t = 16
if t < 12:
Tab print("おはよう")
elif t <= 17:
Tab print("こんにちは")
else:
Tab print("こんばんは")
```

```
#include <stdio.h>
int main(void) {
    int t = 16;
    if (t < 12) {
        printf("おはよう¥n");
    } else if (t <= 17) {
        printf("こんにちは¥n");
    } else {
        printf("こんばんは¥n");
    }
    return 0;
}
```


繰り返し while

- 処理を**繰り返し**実行する。
  ⇒ 条件が**真(YES)**の間，**処理群を繰り返し実行**し，
  偽(NO) になるとループ終端の下にある次の処理を実行する.
- 繰り返し条件を記述するために，
  カウンタの役割を持つ変数を用意することが多い.
  - 処理群の最後でカウントアップするなど，
    処理群の中で値を変更しないと無限ループしてしまう

```
while 条件式:
Tab 処理群
次の処理
```

```
while (条件式) {
    処理群
}
次の処理
```

繰り返し while

- サンプルコード

```
n = 3
a = 0
i = 0
while i < n:
Tab a = a + 2
Tab i = i + 1
print(a)
```

```
#include <stdio.h>
int main(void) {
    int n = 3, a = 0, i = 0;
    while (i < n) {
        a = a + 2;
        i = i + 1;
    }
    printf("%d¥n",a);
    return 0;
}
```


繰り返し for

- 処理を**繰り返し**実行する。

```
for i in range(a, b, c):
Tab 処理群
次の処理
```

```
for (前処理; 条件式; 後処理) {
    処理群
}
次の処理
```

- **range()関数**
  - 処理群を実行するごとに変数の値が
    a ≦ 変数の値 < b の範囲内で c ずつ変化する
    - range(0, 3, 1) → 0, 1, 2　range(3, 0, -1) → 3, 2, 1
  - `range(a)`　　`range(0, a, 1)` と同じ
  - `range(a, b)`　　`range(a, b, 1)` と同じ

終了値 b は含まれない

繰り返し for

- サンプルコード

```
a = 0
for i in range(3):
Tab a = a + 2
print(a)
```

```
#include <stdio.h>
int main(void) {
    int i, a = 0;
    for (i=0; i<3; ++i) {
        a = a + 2;
    }
    printf("%d¥n", a);
    return 0;
}
```


配列（リスト）

- **リスト**　`list = [1, 2, 3]`　　`int list[3] = {1, 2, 3};`
  - Cのような変数宣言は不要
  - リストを空で初期化したい場合　`list = []`
- **リストの操作**
  - リスト list の要素数を調べる　`len(list)`
  - リスト list に要素を追加する　`list.append(追加する値)`
- **リストの要素を表示**（添字は0番目から開始）
  - リスト list の i 番目の要素を表示する　`print(list[i])`
  - リスト list の全ての要素を表示する　`print(list)`
    - 出力例：`[1, 2, 3]`

配列（リスト）

- サンプルコード

```python
list = [1, 2, 3, 4, 5]
sum = 0
for i in range(len(list)):
    sum = sum + list[i]
print(sum)
```

```c
#include <stdio.h>
int main(void) {
    int list[5] = {1, 2, 3, 4, 5};
    int i, sum = 0, len = 5;
    for (i=0; i<len; ++i) {
        sum = sum + list[i];
    }
    printf("%d¥n", sum);
}
```


関数

- 関数宣言・return文

```
def 関数名(引数, …):
Tab 処理群
Tab return 返り値
```

- 原則，関数は呼び出す前に定義する
- Pythonでは，関数の引数としてリストを渡すことができる

```
返り値の型 関数名(引数の型 引数, …) {
    処理群
    return 返り値;
}
```


関数

- サンプルコード

```python
def my_sum(list):
    sum = 0
    for i in range(len(list)):
        sum = sum + list[i]
    return sum

list = [12, 13, 14]
print(my_sum(list))
```

```c
#include <stdio.h>

int sum(int *a, int len) {
    int sum = 0, i;
    for (i=0; i<len; ++i) {
        sum = sum + a[i];
    }
    return sum;
}

int main(void) {
    int a[3] = {12, 13, 14};
    printf("%d¥n", sum(a, 3));
}
```


学習ツールによるプログラムの実行

- 学習ツールで以下のプログラムを打ち込み，実行してください
  - Runボタンを押す or Ctrl+Enter

```python
def max_i(list):
    max_i = 0
    for i in range(1, len(list)):
        if list[i] > list[max_i]:
            max_i = i
    return max_i

list = [34, 12, 45, 98, 59]
print(list[max_i(list)])
```

Tabで半角スペース×4